

"The Complete Guide to pstack Pt. 2" / X

 x.com/poteto/status/2097732320606507506

2026年8月31日

The Complete Guide to pstack Pt. 2
pstack 完整指南（第二部分）

[128K](#)

In my

[last post](#)

, I showed you why verification is the foundation of everything I do with agents, and how to get started creating your own verification skills. The key lesson was that if an agent can't verify its own work, nothing else matters. You remain the bottleneck, and your whole day will be spent babysitting your agents.

在我的

[最后一篇帖子](#)

我向你展示了为什么验证是我与经纪人合作一切的基础，以及如何开始培养你自己的验证技能。关键在于，如果经纪人无法验证自己的工作，其他一切都无从谈起。你仍然是瓶颈，你的所有时间都将耗费在监督你的经纪人上。

But once you have verification working, the next question is: how do you actually figure out what to build?

但是，一旦验证功能正常运行，下一个问题就是：如何才能真正确定要构建什么？

In this post, I am going to walk you through how I do research, planning, prototyping, and architecture with

[pstack](#)

. This is the exact workflow that allows me to ship thousands of PRs a month into production while keeping code quality extraordinarily high.

在这篇文章中，我将带你了解我是如何进行研究、规划、原型设计和架构设计的。

[pstack](#)

正是这种工作流程，让我能够每月向生产环境提交数千个 PR，同时保持极高的代码质量。



Ending the month of August with 2,462 PRs in production
截至 8 月底，共有 2462 个 PR 处于生产状态。

The art of supervising someone smarter than you 如何指导比你聪明的人

Back in the old days of 2024, to make any change in a system, you first needed to read enough of it to build a mental model of what's going on. Depending on the size and complexity of the codebase, this may have taken you anywhere from hours, to even days and months. With a small change, you could get away with maybe only a local understanding of a small subsystem. If you were refactoring the core however, you'd probably need to have a mental model of how the whole thing works in order to do the refactor correctly and effectively.

回到2024年，要想对系统进行任何更改，首先需要阅读足够多的代码，才能构建一个理解系统运行机制的心理模型。根据代码库的规模和复杂程度，这可能需要花费数小时、数天甚至数月的时间。如果是小改动，或许只需对某个小子系统有局部了解即可。但如果要重构核心代码，则需要对整个系统的运行机制有一个清晰的理解，才能正确有效地进行重构。

Agents obviously remove this barrier. You can make changes to systems very easily by just prompting your agent and it will do it, regardless of how much or little you know about the code. But keeping the quality of the code and user experience high is still difficult, especially if you're not already a domain expert who knows what to look for and ask.

代理显然消除了这一障碍。只需向代理发出指令，即可轻松更改系统，无论您对代码了解多少。但要保持代码质量和用户体验的高质量仍然很困难，尤其是在您并非领域专家，不知道该关注什么和该问什么的情况下。

Even though frontier models have gotten very capable, there are still 2 failure modes that I constantly observe:

尽管前沿模型已经非常强大，但我仍然经常观察到两种失效模式：

1. They are unable to fully understand your intent because they're under/poorly specified.
他们无法完全理解你的意图，因为他们的描述不够清晰或不够具体。
2. They don't have enough context on how to do the work correctly.
他们缺乏足够的背景知识，不知道如何正确地完成这项工作。

Both of these problems are related. Using agents well comes down to how well you're able to prime the agent's context window with high quality context. You can certainly write code that works without doing this, but I find that the outcomes and quality are much better when I've done the work to provide my agents with everything they need to do a high quality job. 这两个问题是相关的。能否有效利用智能体，关键在于能否用高质量的上下文信息来充分引导智能体的上下文窗口。当然，你也可以编写出无需这样做也能运行的代码，但我发现，如果我事先为智能体提供完成高质量任务所需的一切信息，最终的结果和质量都会好得多。

In your own words 用你自己的话

Frontier models are very capable coders. While with older models I might have prompted very specifically what I wanted it to do, almost micromanaging them, the latest models are able to write code better than you or I can. So there's a fine balance I want to strike with telling the agent what I want it to achieve, while giving it the freedom to solve it in ways I might not have thought of.

Frontier 模型拥有非常强大的编码能力。虽然对于旧模型，我可能需要非常具体地指示它们要做什么，几乎是微观管理，但最新的模型编写代码的能力甚至超过了你我。因此，我需要找到一个平衡点：既要告诉智能体我想要它实现什么目标，又要给予它足够的自由，让它以我可能没有想到的方式解决问题。

This is the art of supervising someone smarter than you, on a codebase you haven't written yourself, and where humans can no longer fit the entire mental model of the codebase in their head.

这就是指导比你聪明的人，使用你没有自己编写的代码库，并且人类的头脑已经无法完全理解代码库的整个心智模型的艺术。

One technique I like to use is the indirect prompt. Instead of telling the agent exactly what I want, I try to draw it out of the agent instead - in its own words.

我喜欢使用的一种技巧是间接提示。我不会直接告诉经纪人我想要什么，而是尝试引导经纪人用自己的语言表达出来。

For example, when someone reports an issue in Slack, I will often ask the agent to read the thread and restate the problem in its own words before doing anything else.

例如，当有人在 Slack 中报告问题时，我通常会要求客服人员阅读该讨论串，并用他们自己的话重新描述问题，然后再采取其他行动。

For example, I might say:

例如，我可能会说：

```
/poteto-mode read this slack thread. restate in your own words and in plain english  
what you think the underlying issue is
```

```
/poteto-mode 请阅读此 Slack 讨论串。请用您自己的话，并通俗易懂地复述您认为根本问题是什么。
```

This accomplishes three things:

这样做可以达到三个目的：

First, it forces the agent to compress a noisy conversation into a structured problem statement. Second, it lets me catch misunderstandings immediately. If the agent fixates on a red herring in the thread, I can correct it quickly before it starts writing any code.

首先，它迫使智能体将杂乱的对话压缩成结构化的问题陈述。其次，它让我能够立即发现误解。如果智能体在对话中纠缠于无关信息，我可以在它开始编写任何代码之前迅速纠正它。

And third, I haven't potentially led it down the wrong path by stating my own assumptions and hypotheses which could be incorrect or limit what the agent could otherwise achieve.

第三，我没有因为陈述自己的假设和推测而将其引入错误的道路，这些假设和推测可能是错误的，或者限制了代理人原本可以取得的成就。

Building up a mental model

构建心智模型

Asking the agent to restate itself in a way that you can understand is an important part of working with someone that is smarter than you. That was the inspiration for

[/teach](#)

, a skill that helps your agent explain things to you in an intuitive way. I use it whenever I need to make sure my agent is doing something that makes sense to me.

请经纪人用你能理解的方式重新表述，是与比你聪明的人合作的重要环节。这正是……的灵感来源。

[/教](#)

这项技能可以帮助你的经纪人用通俗易懂的方式向你解释事情。每当我需要确保我的经纪人所做的事情对我来说合情合理时，我都会用到它。

Under the hood, /teach calls out to

[/how](#)

and

[/why](#).

在底层，/teach 调用

[/如何](#)

和

[/为什么](#)

。

`/how` traces runtime mechanics. When you ask `/how`, the agent assesses the complexity of the subsystem. If the subsystem spans multiple directories or services, it spawns parallel explorer agents on fast, efficient models like Grok.

`/how` 参数用于追踪运行时机制。当您使用 `/how` 参数时，代理会评估子系统的复杂性。如果子系统跨越多个目录或服务，它会在 Grok 等快速高效的模型上生成并行资源管理器代理。

`/how is virtualization implemented?`

虚拟化是如何实现的？

`/why` investigates motivation and intent. Code tells you what happens. It rarely tells you why someone wrote it that way. When you run `/why`, pstack queries historical evidence across multiple sources in parallel: Git history and PR review comments, Linear tickets, Notion design docs, Slack conversations, Datadog monitors, Sentry errors, code lineage, and analytics warehouse events.

`/why` 命令用于探究动机和意图。代码只能告诉你发生了什么，却很少告诉你为什么会这样写。运行 `/why` 命令时，pstack 会并行查询多个来源的历史证据：Git 历史记录和 PR 审查评论、Linear 工单、Notion 设计文档、Slack 对话、Datadog 监控、Sentry 错误、代码沿袭以及分析仓库事件。

`/why are we still stuck an old version of node.js?`

为什么我们还在使用旧版本的 node.js？



I use `/teach` whenever I want the agent to restate something so I can better understand and trust its work.

我会在希望代理复述某些内容以便更好地理解 and 信任其工作时使用 `/teach` 命令。

`/teach me why you implemented it this way and not <other way>. what were the tradeoffs you made and why?`

请告诉我你为什么这样实现而不是那样实现。<other way> 你做出了哪些权衡取舍？为什么？

In practice, I have also found that the research done by the `/teach` skill is not just useful to humans, but for agents as well. Even with the latest frontier models, (this also depends on the quality of the harness), in general I find that they still often state things confidently without backing it up with data or actually reading the code needed to build up a mental model of how it works. So this act of teaching you what it's going to do and why ends up helping the agent too.

在实践中，我发现 `/teach` 技能所做的研究不仅对人类有用，对智能体也同样有用。即使是最前沿模型（这也取决于框架的质量），我发现它们通常仍然会自信地陈述一些事情，却缺乏数据支撑，也没有真正阅读构建其工作原理所需的代码。因此，这种教你它要做什么以及为什么这样做的行为，最终也会帮助到智能体。

Learning from history

从历史中吸取教训

Many of my projects span multiple conversations. For example, a few months ago I was working on fixing virtualization bugs and perf issues that people were reporting in Cursor. I realized that every time I started a new chat I had to basically start over with building up the rich context my agent had before when it was solving a similar problem.

我的许多项目都涉及多个对话。例如，几个月前，我正在修复用户在 Cursor 中报告的虚拟化错误和性能问题。我意识到，每次开始新的聊天时，我基本上都必须重新构建代理之前解决类似问题时所掌握的丰富上下文信息。

What I realized is that your past transcripts are often a gold mine for rich context. pstack ships with the

[/recall](#)

skill to pull your recent context from chat history, so even fresh agents have the right context they need to get back to a good state.

我意识到，你过去的成绩单往往蕴藏着丰富的背景信息，是一座宝库。pstack 自带.....

[/记起](#)

能够从聊天记录中提取最近的上下文信息，因此即使是新入职的客服人员也能获得所需的正确上下文信息，从而迅速进入良好状态。

```
/recall the work i did yesterday on virtualization and then read this bug report on slack
```

回想起昨天我在虚拟化方面所做的工作，然后又在 Slack 上看到了这个 bug 报告。

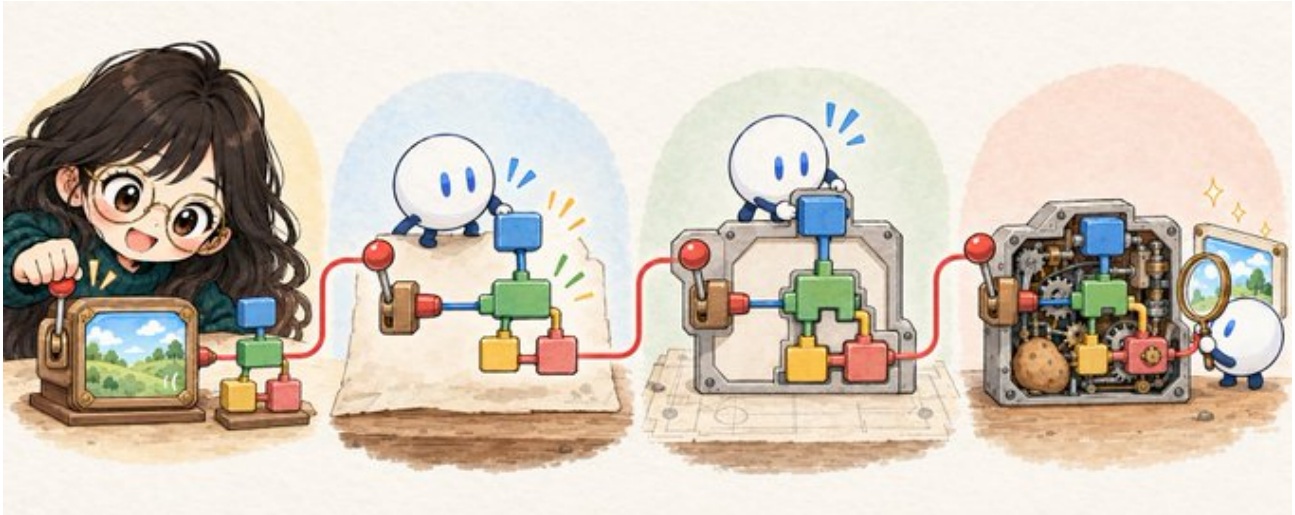
Using `/teach`, `/recall`, `/how`, and `/why` are how I keep my own mental models of the codebase up to date, compressed into a form I can easily understand and remember. And, it helps agents too!

我使用 `/teach`、`/recall`、`/how` 和 `/why` 命令来更新我对代码库的理解，并将这些理解压缩成易于理解和记忆的形式。而且，这对智能体也很有帮助！

Working backwards 倒推

Once you understand the problem, how do you specify the solution?

一旦你理解了问题，你如何提出解决方案？



In my opinion, most harnesses that have plan modes tend to over-specify implementation details and under-specify everything else. That's why in pstack, I cheekily said that

["I don't believe in planning"](#)

. The truth is that I do plan, but I do so through code.

在我看来，大多数带有计划模式的框架往往过度强调实现细节，而忽略其他方面。这就是为什么我在 pstack 中开玩笑说.....

[我不相信计划。](#)

事实上，我的确会做计划，但我是通过代码来实现的。

For certain kinds of work, like creating shared code or packages that others will use, I am a big believer in readme driven development. If you're not familiar with it, it's a technique of development that was popular back in the day, where you start with crafting your readme first. This forced you to put on your developer experience hat, where you start with describing the APIs to a hypothetical user, and work backwards to the implementation and architecture.

对于某些类型的工作，例如创建共享代码或供他人使用的软件包，我非常推崇基于 README 的开发方式。如果你不熟悉这种方式，它是一种曾经很流行的开发技巧，即先编写 README 文件。这迫使你从开发者的角度出发，首先向假想用户描述 API，然后反向推导出实现和架构。

For example, when I was building Dune, our in-house client framework for desktop apps, I started by first writing a tutorial for it, so I could understand what it would be like to build an app with it. Or at least I tried to. It was real a struggle getting the agent to produce anything good or readable. So I had to first spend some time sharpening my knife, by creating the [/technical-writing](#)

skill.

例如，当我构建我们内部用于桌面应用程序的客户端框架 Dune 时，我首先编写了一个教程，以便了解用它构建应用程序会是什么样子。或者至少我是这么尝试的。要让代理生成任何好用或易读的代码真是太难了。所以我不得不先花些时间磨练自己的技能，创建……

[技术写作](#)

技能。

The first pass of the readme without the /technical-writing skill was painful to read because it mixed up different goals. It tried to be a tutorial, a how-to guide, an architectural explanation, and an API reference all in the same document, written with your usual AI slop and

[mannered prose](#)

初稿的 README 文件缺乏技术写作技巧，读起来十分痛苦，因为它混淆了不同的目标。它试图在一份文档中同时扮演教程、操作指南、架构说明和 API 参考的角色，而且文笔粗糙，充满了人工智能常见的错误。

[文风优美的散文](#)

。



/technical-writing uses the

[Diátaxis framework](#)

to separate documentation into four distinct modes:

/technical-writing 使用

[对话框架](#)

将文档分为四种不同的模式：

1. Tutorial: Learning by doing. A lesson that leads a newcomer through a series of steps to build something visible.

教程：边做边学。本课程引导新手通过一系列步骤构建出可见的成果。

2. How-to guide: Steps to solve a specific, real-world problem for an experienced user.

操作指南：为经验丰富的用户解决特定实际问题的步骤。

3. Reference: Dry, complete, authoritative technical descriptions of machinery, APIs, and configuration flags.

参考资料：对机械、API 和配置标志的干涩、完整、权威的技术描述。

4. Explanation: High-level discussion that clarifies and illuminates background, design choices, and tradeoffs.

说明：高层次的讨论，阐明和阐明背景、设计选择和权衡取舍。

It also uses

[/unslop](#)

, so it produces documentation that is very readable.

它还使用

[/unslop](#)

因此，它生成的文档非常易读。

Writing a plan this way is very helpful because it also gives your agents a concrete target and goal that it can check its own work against. And of course, it's also much easier to understand what exactly the agent is going to build.

以这种方式制定计划非常有帮助，因为它能为你的代理人提供一个具体的目标，让他们可以以此来检验自己的工作。当然，这样也更容易理解代理人究竟要构建什么。

Many of pstack's skills compound here in the design phase. For example:

pstack 的许多技能在设计阶段都得到了充分运用。例如：

```
(1) /recall my work fixing virtualization bugs and perf issues from the past 7 days. use /how and /why to understand how our current virtualization implementation works. (2) then use /poteto-mode planning and /technical-writing to come up with a new virtualization engine that categorically eliminates flickering and jittering. let's start by writing a tutorial on how i would use this new package to virtualize a React app (3) after you write the plan, /teach me and prove to me why this new approach is superior to our current engine
```

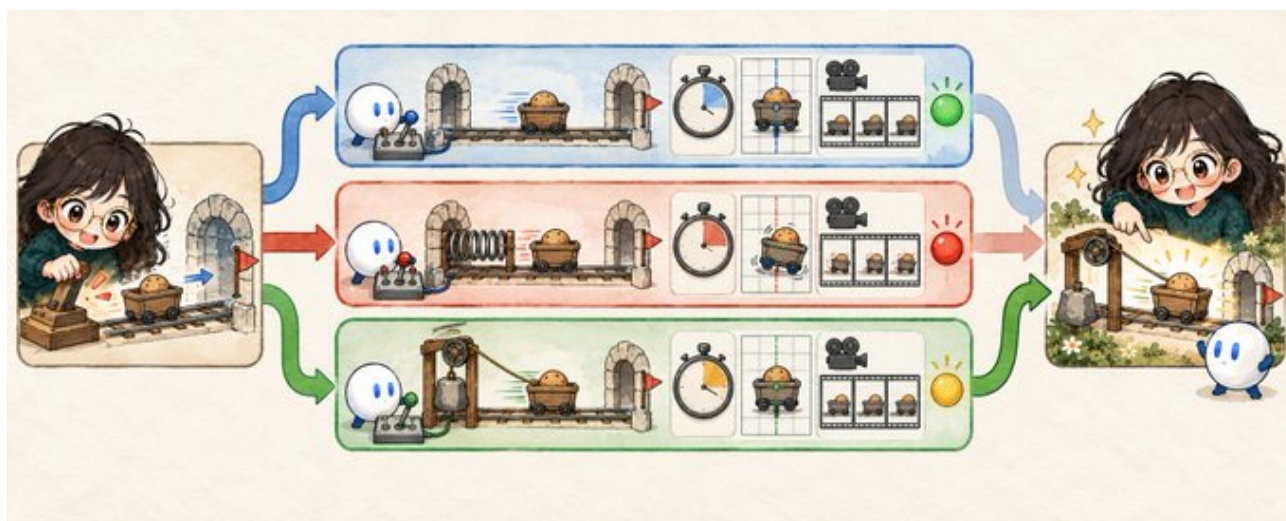
```
(1) /回顾我过去 7 天修复虚拟化错误和性能问题的工作。使用 /how 和 /why 来了解我们当前的虚拟化实现是如何工作的。(2) 然后利用 /poteto-mode 规划和 /technical-writing 来开发一种全新的虚拟化引擎，彻底消除闪烁和抖动。我们先来写一篇教程，介绍如何使用这个新包来虚拟化一个 React 应用。(3) 写完计划后，请教我并向我证明为什么这种新方法优于我们目前的引擎。
```

The technique here is really about drawing out interesting and rich context that gives your agents the ability to see the problem the same way you do - not just as a small slice:

这里的技巧在于挖掘出有趣且丰富的背景信息，使你的代理人能够像你一样看待问题，而不仅仅是看到问题的一小部分：

1. The first part of the prompt recalls relevant past and present context about how virtualization is implemented in my app.
提示的第一部分回顾了与我的应用程序中虚拟化实现方式相关的过去和现在的背景信息。
2. The second part guides the agent to use that context, such as bugs it has fixed before, to come up with a new design that eliminates those problems entirely.
第二部分指导代理利用上下文（例如之前修复的错误）来提出一种新的设计，从而彻底消除这些问题。
3. The final piece is asking your agent to prove to you that this new package is superior. This is where high quality tools like [verification skills](#) are important to have.
最后一步是请你的经纪人向你证明这个新套餐更胜一筹。这时，像.....这样的高质量工具就派上用场了。
[验证技能](#)
拥有它们很重要。

Measure a hundred times, cut once 量一百遍，切一次



When planning, two of the most common mistakes I see are:
在制定计划时，我最常看到的两个错误是：

1. Accepting the agent's first design.
接受代理人的第一个设计。
2. Overcooking the plan without empirical evidence.
计划制定得过于复杂，缺乏实证依据。

When humans wrote code, we often collaborated with each other over design documents. These were docs that talked about high level architecture, alternatives considered, tradeoffs, and any unusual implementation notes. It was very common to go through multiple iterations of these docs before you landed on a settled design.

人类编写代码时，经常会围绕设计文档进行协作。这些文档阐述了高层架构、备选方案、权衡取舍以及任何特殊的实现细节。在最终确定设计方案之前，反复修改这些文档是很常见的。

With agents, while we can skip the ceremony of the design doc, I often see the mistake of accepting the first thing the agent gives back to you. With pstack, we can instead take the "measure twice, cut once" approach to its limit, using parallel agents.

使用智能体虽然可以省去设计文档的繁琐步骤，但我经常看到人们犯的错误是直接接受智能体返回的第一个结果。而使用 pstack，我们可以利用并行智能体，将“三思而后行”的原则发挥到极致。

In pstack, playbooks aren't skills, but reference files inside of /poteto-mode. These playbooks are conditionally loaded (for token efficiency) depending on the type of task you're working on. These 23 playbooks (as of 0.15.0) each contain a workflow that I use when I'm doing a task.

在 pstack 中，playbook 不是技能，而是位于 `/poteto-mode` 目录下的引用文件。这些 playbook 会根据你正在处理的任务类型进行条件加载（以提高令牌效率）。截至 0.15.0 版本，这 23 个 playbook 分别包含我执行任务时使用的工作流程。

Unlike skills, playbooks are automatically used by the agent as part of /poteto-mode. For example:

与技能不同，战术手册会在 /poteto-mode 命令下由智能体自动使用。例如：

```
/poteto-mode prototype a few options for the new dropdown menu /poteto-mode fix  
this bug /poteto-mode eval this skill change  
/poteto-mode 原型，为新的下拉菜单提供了一些选项 /poteto-mode 修复此错误  
/poteto-mode 评估此技能变化
```

Prototyping is one of my favorite pstack playbooks. It gives you many attempts at a goal and helps the agent reason about the best option. This is useful not just for visual prototyping, but also prototyping different solutions for features, bug fixes, and so on.

原型设计是我最喜欢的 pstack 策略之一。它允许你多次尝试实现目标，并帮助智能体推理出最佳方案。这不仅适用于视觉原型设计，也适用于为功能、错误修复等设计不同的解决方案原型。

```
/poteto-mode prototype a few options for <feature request>. use /control-app* and  
take videos/screenshots for me to review and choose from  
/poteto-mode 原型提供了一些选项<feature request>请使用 /control-app* 命令并录制  
视频/截屏，以便我审核和选择。
```

* note: `/control-app` is the verification skill we created in

[Part 1](#)

* 注意：`/control-app` 是我们创建的验证技能

[第一部分](#)

When prototyping visual changes, the agent builds throwaway sketches in your app or in a scratch directory. If it is testing a UI interaction, it puts two or three variations behind a simple switcher. Then it drives the interaction with the `/control-app` skill, takes screenshots of each variant, and measures the actual timing or layout.

在进行视觉变更原型设计时，代理会在你的应用或临时目录中构建临时草图。如果它正在测试用户界面交互，它会在一个简单的切换器后面放置两到三个不同的版本。然后，它会使用 ``/control-app`` 技能来驱动交互，截取每个版本的屏幕截图，并测量实际的响应时间和布局。

Prototyping is planning, but with code. It allows agents the freedom to explore the problem space, and to give them a chance to surprise you with something you wouldn't have thought of yourself. Prototypes allow agents to answer their own questions with empirical evidence instead of waiting for my input.

原型设计就是用代码进行规划。它赋予用户探索问题空间的自由，并让他们有机会提出你意想不到的方案。原型设计让用户能够利用经验证据来回答自身的问题，而不是等待我的输入。

Architecting bigger changes

构建更大的变革

As an engineer in the agentic era, it's more important to spend my time on architecture, choosing the right data structures, and thinking about how the systems I build will work together. My agents fill in the implementation details.

作为智能体时代的一名工程师，我更应该把时间花在架构设计上，选择合适的数据结构，以及思考我构建的系统如何协同工作。我的智能体负责填充实现细节。



Another useful skill that pstack ships with is

[/architect](#)

. It structures design into distinct, disciplined phases:

pstack 自带的另一个实用技能是

[/建筑师](#)

它将设计过程划分为清晰、严谨的阶段：

1. Ground the problem. The agent runs /how and /why over the affected systems to build an accurate mental model of existing ownership and constraints.
找出问题根源。代理程序会分析受影响的系统，构建一个关于现有所有权和约束条件的准确心理模型。
2. Sketch. The agent enters an architecture arena. It spawns independent candidate runners in parallel, often across different model families. Each runner receives the grounding brief and drafts a complete design package: the caller's usage sketch, the core type definitions, public function signatures, and a concise rationale. These are usually done by sketching out just the type signatures, that derive from how we want call sites to look like. Each runner must evaluate interface depth, examine failure modes on weak models, and screen against our catalog of design red flags.
草图。代理进入架构领域。它并行生成独立的候选运行器，这些运行器通常跨越不同的模型族。每个运行器都会收到基础概要，并起草完整的架构设计包：调用者的使用草图、核心类型定义、公共函数签名以及简明的原理说明。这些通常是通过绘制类型签名草图来完成的，类型签名源于我们希望调用点呈现的形式。每个运行器都必须评估接口深度，检查弱模型上的故障模式，并对照我们的设计风险清单进行筛选。
3. Cross-judge and Synthesize. A cross-judge agent using a different model than the main agent evaluates the candidates against a strict rubric.
交叉评判与综合。一个使用与主评判者不同模型的交叉评判者，根据严格的评分标准对候选人进行评判。
4. Implement against the sketch. The agent replaces the sketch's placeholder bodies with real logic. If the agent discovers during implementation that a function needs unexpected parameters or extra state, it surfaces the discrepancy.
根据草图进行实现。智能体会将草图中的占位符主体替换为实际逻辑。如果在实现过程中，智能体发现某个函数需要意外的参数或额外的状态，它会将这种差异反映出来。
5. Scrap when the design is wrong. If during implementation we find that the sketches were wrong, the agent throws it all away and starts over.
设计出错时就放弃。如果在实施过程中发现草图有误，系统会将所有工作推翻重来。

The point here is to give the agent a self contained mini-loop where it can synthesize multiple competing designs from different model families into one optimal approach, and take care to be rigorous and not be afraid to throw its design away if it turns out that the architecture it came up with is wrong based on empirical proof. If the same workaround appears across unrelated call sites, or if the types require escape hatches like `any` or forced casts, that is empirical proof that the architecture is wrong.

关键在于为智能体提供一个独立的微型循环，使其能够将来自不同模型族的多个竞争设计方案综合为一个最优方案。同时，务必严谨，如果基于经验证明其提出的架构存在错误，则应毫不犹豫地放弃该设计。如果相同的变通方案出现在不相关的调用点，或者类型需要像 any 这样的逃生通道或强制类型转换，则这便是架构错误的经验证据。

```
/architect this new <feature request>  
/architect 这个新<feature request>
```

The big lesson here is that it's far more effective to plan with code using /poteto-mode prototyping and /architect.

这里最重要的教训是，使用 /poteto-mode 原型设计和 /architect 进行代码规划要有效得多。

It's also why I never bother with reviewing abstract plans adversarially. The agents start hallucinating theoretical risks, and inventing complex edge cases to protect against problems that will never happen. Don't overcook your plans when they're still abstract: let the agent answer open questions on its own through prototyping and verifying its own work.

这也是我从不费心费力地以对抗的方式审查抽象计划的原因。智能体会开始臆想理论上的风险，并捏造复杂的极端情况来防范永远不会发生的问题。不要在计划还处于抽象阶段时就过度完善：让智能体通过原型设计和验证自身工作来解答开放性问题。

Okay but I really want a planning doc
好的，但我真的想要一份规划文件。



While pstack doesn't come with a planning skill, it does ship with a [multi-phase planning playbook](#)

. I typically use this after the agent has come up with a design I'm happy with, as a way to create a tactical execution plan.

虽然 pstack 本身不具备规划技能，但它确实附带了.....

多阶段规划手册

我通常在代理商提出令我满意的设计方案后使用这种方法，以制定战术执行计划。

```
/poteto-mode turn this design into a plan
/poteto-mode 将此设计转化为计划
```

Every single task in the plan is structured around proof and verification. The playbook tells agents that tests alone are not sufficient verification. It's verified only when it has actually run the code and verified that it works.

计划中的每一项任务都围绕着证明和验证展开。操作手册告诉代理，仅靠测试不足以进行充分验证。只有实际运行代码并验证其有效后，才能算作验证。

Every plan is checked by an automated script that validates its structure and formatting. Once approved, the plan executes item by item. Each PR is small, self-contained, and easily reviewed.

每个计划都会经过自动脚本检查，以验证其结构和格式。一旦获得批准，计划就会逐项执行。每个 PR 都很小巧、独立且易于审查。

For really large projects (like one that might take me a whole week), I may sometimes decide to commit the plans temporarily to the codebase so that other agents are aware of the work in progress. But I typically delete them when I'm done so I don't leave the codebase in a state of confusion. I don't find it valuable to keep plans around permanently. 对于一些非常大的项目（比如可能需要我花一整周时间完成的项目），我有时会决定暂时将计划提交到代码库中，以便其他成员了解正在进行的工作。但我通常会在完成后删除它们，以免代码库陷入混乱。我认为永久保留计划并没有什么意义。

The workflow in practice

实际工作流程

To see how all of these pieces fit together, let us walk through three concrete examples of how I prompt these workflows.

为了了解所有这些部分是如何组合在一起的，让我们来看三个具体的例子，了解我是如何引导这些工作流程的。

Example 1: Researching an ambiguous bug

示例 1：调查一个含义模糊的漏洞

When an issue appears in production and the root cause is unclear:

当生产环境中出现问题且根本原因不明时：

```
/poteto-mode investigate why background workers periodically fail with timeout
errors. give me a breakdown of what we know, what data you used, and your best
```

hypotheses.

/poteto-mode 请调查后台工作进程为何会周期性地出现超时错误。请详细说明我们已知的信息、您使用的数据以及您最佳的假设。

The agent explores the code, checks metrics and historical commits in parallel, and gives you its best educated guesses on where the problem might lie.

该代理程序会并行地探索代码、检查指标和历史提交记录，并给出它认为最合理的猜测，指出问题可能出在哪里。

Example 2: Designing a new service boundary

示例 2：设计新的服务边界

When introducing a new subsystem that other modules will depend on:

引入其他模块所依赖的新子系统时：

/poteto-mode we need to add rate limiting for external webhooks. /architect this first, and answer any open questions with prototypes. let me review before proceeding.

/poteto-mode 我们需要为外部 webhook 添加速率限制。/architect 请先构建这个架构，并用原型解答任何未决问题。让我先审核一下再继续。

The agent grounds the existing webhook architecture, spins up competing design runners across multiple models, benchmarks with throwaway prototypes, and produces a clean, verified interface.

该代理程序基于现有的 webhook 架构，启动多个模型的竞争设计运行器，使用一次性原型进行基准测试，并生成一个清晰、经过验证的界面。

Example 3: Executing a multi-PR migration

示例 3：执行多 PR 迁移

When executing a complex refactor across many files:

当对多个文件执行复杂的重构时：

/poteto-mode create a plan to migrate our entire UI library to StyleX. break the migration into small, verifiable PRs. each PR must have its visual regression tests and live verification steps. i want the final result to be 100% identical compared to the original - bugs included

/poteto-mode 制定一个计划，将我们整个 UI 库迁移到 StyleX。将迁移工作分解成若干个小的、可验证的 PR。每个 PR 都必须包含视觉回归测试和在线验证步骤。我希望最终结果与原始版本 100% 相同——包括所有 bug。

The agent breaks the work into independent steps, writes an auditable checklist, and prepares each unit so that it can be built, verified, and landed safely.

该代理人将工作分解成独立的步骤，编写可审计的检查清单，并准备好每个单元，以便能够安全地建造、验证和着陆。

Example 4: Fix stuff people report on Slack

示例 4：修复用户在 Slack 上反馈的问题

If you've ever seen me on Slack in one of our issues or feedback channels, you'll probably have seen these classics:

如果你曾在 Slack 的问题或反馈频道中见过我，你可能已经看过这些经典内容：

```
# thread already has sufficient context /poteto-mode do it /poteto-mode repro this  
with /control-app. if it repros on main, fix it and show me a video as proof  
# 帖子已有足够的上下文 /poteto-mode 执行 使用 /poteto-mode 和 /control-app 重现  
此问题。如果主程序也能重现，请修复并提供视频作为证明。
```

Many of the skills I've talked about here are already automatically used by /poteto-mode, so the vast majority of times you can just use /poteto-mode and move on with your life! 我在这里提到的许多技能都已经被 /poteto-mode 自动使用了，所以绝大多数情况下，你只需要使用 /poteto-mode 就可以继续你的生活了！

The art of planning

规划的艺术

Plan Mode is often used as a way to convince yourself that the agent is going to do the right thing. But the reality is that abstract plans only give you the illusion of progress. A long and lengthy plan makes it look like you and your agent were very productive, but it's probably lacking in substance.

计划模式常常被用来说服自己代理人会做出正确的决定。但实际上，抽象的计划只会给人一种进展顺利的错觉。冗长的计划会让你和代理人看起来效率很高，但实际上可能缺乏实质内容。

pstack gives you tools to combine thorough investigation, empirical evidence, and rigorous verification. When you plan this way, engineering with agents stops feeling like a gamble. It becomes predictable and repeatable.

pstack 提供了一系列工具，可以将深入的调查、经验证据和严格的验证结合起来。当你以这种方式进行规划时，基于智能体的工程设计就不再像是一场赌博，而是变得可预测且可重复。

Thanks for reading, and stay tuned for Part 3!

感谢阅读，敬请期待第三部分！